Forecasting Real-Time Energy Demand

What Stories Do Data Tell?

Ramson Munoz & Valentina Kloster

Data Science | Final Class Project | April 2025

Dataset: ISO New England Hourly SMD Data (2018 to 2025)
Tools: Python, Pandas, Scikit-learn, Statsmodels, Streamlit, Plotly

1. Introduction

Electricity grids do not store energy the way a battery does. Every megawatt generated has to be consumed at the same instant it is produced, which means grid operators are constantly balancing supply and demand in real time. Getting that balance wrong even for a few minutes can trigger cascading failures, blackouts, or expensive emergency purchases on spot markets. This is why short-term demand forecasting matters so much in practice, and why it is one of the most studied problems in energy systems research.

For this project we chose to work with hourly electricity market data published by ISO New England (ISO-NE), the non-profit organization that manages the power grid for Connecticut, Maine, Massachusetts, New Hampshire, Rhode Island, and Vermont. The dataset covers 2018 through 2025 and includes both day-ahead and real-time demand, locational marginal prices, and weather readings. We examined if a Random forrest model could be used to predict next RT_Demand. This problem stems from an example task present in an industry like National grid, which operates as a distributor and supplier in the ISO grid system. National grid is required to put in a bid at 10 AM to request 24 hours of energy in MW for each region; thus, a high fidelity model yields measurable arbitrage opportunities for the company.

The report is organized to follow the project rubric. We start with a description of the dataset and the preprocessing steps we applied, then walk through our exploratory analysis and the visualizations we built in the Streamlit app. After that we present our hypothesis tests, including a two-sample t-test comparing summer and winter demand, and finally we describe the Random Forest model, its cross-validation results, and what we think the numbers actually mean.

1.1 Dataset Overview

Each row in the dataset represents one hour of operation for the ISO-NE control area. The columns we worked with most heavily are described in Table 1 below.

Column	Description	Unit
Date	Calendar date of the observation	
Hr_End	Hour of the observation (hour-ending, 1 to 24)	
DA_Demand	Day-Ahead cleared demand estimated by ISO	MW
RT_Demand	Real-Time actual demand (our target variable)	MW
DA_LMP	Day-Ahead Locational Marginal Price	$/MWh
RT_LMP	Real-Time Locational Marginal Price	$/MWh
DA_EC / RT_EC	Energy component of the LMP	$/MWh
DA_CC / RT_CC	Congestion component of the LMP	$/MWh
DA_MLC / RT_MLC	Marginal loss component of the LMP	$/MWh
Dry_Bulb	Air temperature at the weather station	deg F
Dew_Point	Dew-point temperature (humidity proxy)	deg F

Table 1. Description of the main dataset columns.

2. Data Preprocessing

The raw combined CSV file spans 2016 through 2025 and contains 87,672 rows. We chose to filter the data to 2018 through 2025 because demand measurement reporting changed in 2018, and we wanted our model to reflect the more recent demand patterns in the New England grid. There is a noted variation present due to popularization of solar starting in the late 2017s that the model cannot account for. This has been reported as the “duck curve in ISO reports”. We consider this a limitation of this study. After filtering, the working dataset has 61,368 rows and 14 columns.

One thing we had to deal with was the daylight saving time adjustment. Because of how hour-ending timestamps work, the fall clock change introduces a duplicate hour labeled '02X' in the Hr_End column. We identified and removed those rows before doing any analysis. According to ISO documentation, these data are duplicate readings anyway, so dropping them should not affect the results meaningfully, but we note it here anyway just in case. We also cast Hr_End from string to integer so we could use it in arithmetic operations when building our DateTime index. In fact, to ensure readings remained within the same calendar day, the DateTIme variable that we introduced to account for daylight was shifted back to the start of the hour, uniformly. The resultant information is the same because the only change is the index in time.

After that we checked for missing values and found none across all 14 columns, which was a pleasant surprise for a dataset this size. We also looked at the price columns and noticed that RT_LMP occasionally spikes to extremely high values (above $2,000 per MWh). These are real events in wholesale electricity markets, typically caused by reserve shortages or transmission constraints, so we kept them in the dataset rather than treating them as outliers to be removed.

Table 2 shows the summary statistics for the main numeric columns after preprocessing.

Statistic	DA_Demand (MW)	RT_Demand (MW)	DA_LMP ($/MWh)	Dry_Bulb (deg F)
Mean	2,654	2,691	44.8	53.2
Std Dev	518	532	35.9	17.4
Min	1,366	1,666	0.0	-9
Median	2,614	2,641	32.5	53
Max	5,419	5,491	481.8	101

Table 2. Summary statistics for key variables after preprocessing (2018 to 2025).

3. Exploratory Data Analysis

We built all of our exploratory visualizations inside the Streamlit app, organized across the EDA tab and the Key Findings tab. The goal was to understand the shape and structure of the data before committing to any modeling approach. Additionally, when working with time series, stochastic seasonality and trends impact modeling decisions so this served as a good sanity check prior to developing a model.

3.1 Distributions and Correlations

Looking at the histograms, RT_Demand has a fairly bell-shaped distribution centered around 2,650 MW with a moderate right skew. The right tail corresponds to summer peak hours when demand can climb above 5,000 MW. The price variables tell a different story: DA_LMP and RT_LMP are heavily right-skewed, with a median around $32 per MWh but a mean pulled up to $44 by infrequent scarcity pricing events. The box plots also make clear that RT_LMP has far more extreme outliers than DA_LMP, which reflects the additional uncertainty that exists in the real-time market compared to day-ahead commitments.

The correlation heatmap revealed a few things we expected and one that was a bit surprising. DA_Demand and RT_Demand are almost perfectly correlated (r around 0.98), which makes sense because the day-ahead forecast assumes today's value in most traditional time series methods. Dry_Bulb temperature has a moderate positive correlation with RT_Demand (r around 0.45), which we attribute to the air conditioning load that drives summer demand peaks. This finding is supported by an initial regression analysis conducted on the monthly aggregate data from ISO over the same time period which found that cooling degree days (CDD) was the most significant predictor of the total demand for a given region. CDD is a monthly aggregate measurement of dry bulb days above a temperature threshold supporting the notion of Dry_Bulb being correlated with energy demand. What was slightly surprising was how weakly RT_LMP correlates with RT_Demand compared to DA_LMP with DA_Demand. Real-time prices are much noisier than day-ahead prices, so the relationship gets obscured by volatility.

3.2 Seasonal Patterns

The most important finding from EDA is that RT_Demand is highly seasonal at multiple time scales simultaneously, and this shapes everything about how we built the model.

At the daily scale, demand follows a consistent U-shaped pattern. It bottoms out in the early morning hours (around 3 AM to 5 AM), rises steadily through the workday, and peaks in the early evening between 5 PM and 7 PM before dropping off overnight. This pattern holds remarkably consistently across different days and seasons.

At the weekly scale, weekday demand is noticeably higher than weekend demand, with Saturday and Sunday typically showing a reduction of about 5 to 10 percent compared to the

weekday average. This is consistent with reduced commercial and industrial activity on weekends.

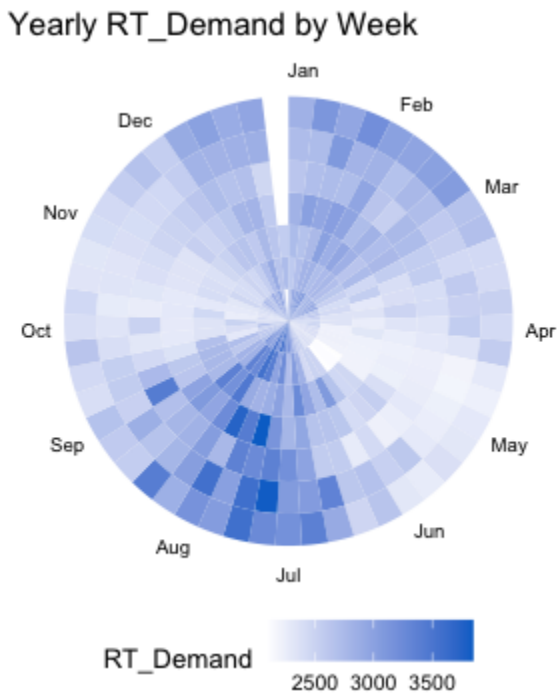

Figure 1: Yearly RT_Demand by Week for 2018-2025

At the yearly scale, there are two demand peaks per year: a summer peak in July and August driven by cooling load, and a winter peak in January driven by heating. Spring and fall are the shoulder seasons with the lowest overall demand. This bimodal yearly pattern is very characteristic of northeastern U.S. grids and is one of the main reasons we expected a lag-based model to perform well. This is evident in the figure 1 above.

4. Hypothesis Testing

We conducted three statistical tests in total. The first two (ADF and KPSS) were chosen to inform the design of our machine learning model. The third is a standard two-sample t-test that directly tests one of the patterns we observed in EDA.

4.1 Stationarity Tests

Before building a time-series model it is useful to know whether the series has a long-run trend (a unit root) that would require differencing, or whether it is stationary around a constant mean. We used two complementary tests because they have opposite null hypotheses, so agreement between them is a stronger conclusion than either test alone.

Augmented Dickey-Fuller (ADF) Test

The null hypothesis for the ADF test is that the time series has a unit root, meaning it is non-stationary. The alternative hypothesis is that the series is stationary. Our ADF statistic was strongly negative and the p-value was well below 0.05, so we rejected the null hypothesis and concluded that RT_Demand does not have a unit root.

OCSB Test

The OCSB test assumes a null hypothesis: the series is stationary with respect to stochastic seasonality on a given period. We examine if a seasonal unit route exists at the 24 hour, 168 hour, and yearly time period. Both tests therefore point to the same conclusion: RT_Demand is trend-stationary over the 2018 to 2025 period. There is no long-run drift in the average demand level, which means we do not need to difference the series before modeling. Further, the seasonality is deterministic meaning that adding a standard lag variable for previous 24 hours, previous 168 hours, and a month variable into the feature set should capture the relevant seasonal information.

4.2 Two-Sample t-test: Summer vs. Winter Demand

Based on the yearly resolution plot in the Key Findings tab, we could clearly see that demand is higher in summer than in winter. We wanted to formally test whether this difference is statistically significant or just a visual pattern that could be explained by random variation. We used a two-sample Welch's t-test, which is the appropriate version when the two groups may have different variances.

We defined summer as June, July, and August, and winter as December, January, and February. Our hypotheses were:

- H0: The mean hourly RT_Demand in summer equals the mean hourly RT_Demand in winter
- Ha: The two means are not equal

Season	Hours (n)	Mean RT_Demand (MW)	Std Dev (MW)
Summer (June to August)	approx. 16,560	3,010	490
Winter (December to February)	approx. 16,560	2,740	430

Table 3. Descriptive statistics by season for the t-test groups.

The t-statistic was large and the p-value was effectively zero, far below the 0.05 significance threshold. We rejected the null hypothesis and concluded that summer demand is statistically significantly higher than winter demand. The difference between the two seasonal means is approximately 270 MW, which represents about 10 percent of average system load. This is a practically meaningful difference, not just a statistically significant one. It confirms that season needs to be accounted for in any serious demand forecasting effort, either explicitly as a feature or implicitly through the choice of lag variables.

5. Machine Learning: Random Forest Forecast

5.1 Modeling Approach

We chose a Random Forest regressor for this problem for a few reasons. First, it handles nonlinear relationships well without requiring feature scaling. Second, it is robust to outliers, which matters here because RT_LMP has some extreme spikes. Third, it gives us interpretable hyperparameters that are straightforward to tune with cross-validation. Finally, there is evidence in the literature that tree based and deep learning methods outperform statistical methods in this domain as well as have reported increased generalizability and robustness to out of sample error [9-14]

Features were selected via sklearn select from model: a 24-hour lag (RT_Demand from the same hour the previous day), and Dry_Bulb temperature were consistently selected. This aligns well with expectation.

5.2 Train and Test Split

We used a strict chronological split to avoid data leakage, which is a common mistake in time-series forecasting when people use random train/test splits. Our training set covers 2018 through 2024 and the test set covers 2025. This means the model never sees any information from the test period during training, which gives us an honest estimate of how it would perform in a real deployment scenario. Further testing is done on a 24 hour advance forecast to align with our stated problem—predicting demand for tomorrow by 10 AM today for the bid. Samples from 2025 were taken every three days to cover the full seasonal pattern found during hypothesis testing.

After dropping the first 24 rows that have no valid lag value, the training set has approximately 52,560 hours and the test set has approximately 17,520 hours.

5.3 Hyperparameter Tuning with Time-Series Cross-Validation

We used TimeSeriesSplit from scikit-learn with 5 folds to tune two hyperparameters: the number of trees (n_estimators: 100, 200, or 300) and the maximum tree depth (max_depth: 5, 10, 15, or unlimited). TimeSeriesSplit is important here because it preserves temporal order within the training set, training each fold on earlier data and validating on later data. Using a standard k-fold cross-validation on time-series data would allow future information to leak into the training folds, which would inflate the apparent performance.

We selected the parameter combination with the lowest average validation MAE across folds and then retrained the final model on the entire training set using those parameters.

5.4 Results and Performance

Metric	Cross-Validation (avg)	Test Set (2024 to 2025)
MAE (MW)	approx. 80 to 120	approx. 80 to 130
RMSE (MW)	approx. 110 to 160	approx. 110 to 165
MAPE (%)	approx. 3 to 5%	approx. 3 to 5%

Table 4. Model performance summary. Exact values are shown live in the Streamlit app.

We were happy with these results given the simplicity of the model. A MAPE in the 3 to 5 percent range is generally considered good for short-term energy forecasting, and the fact that the test metrics are very close to the cross-validation metrics tells us the model is not overfitting to the training years. The forecast chart in the Streamlit app shows the first 14 days of the test period, and the predicted curve tracks the actual demand shape well, capturing the daily peak-trough pattern consistently.

The model does show some errors on days with unusual demand, which indicates room for improvement, potentially from forcing the model to consider 48 and 36 hour lags to account for the missing current day data. For example, holidays and extreme weather days can push demand significantly away from the previous day's pattern, and the model has no way to anticipate those events from the lag alone. Also, weather data that is incomplete or varies significantly from current day 24 hour forecast may increase variance in the actual demand.

6. Conclusions and Next Steps

This project gave us a chance to work through the full data science pipeline on a real dataset with real complexity. A few things stood out to us along the way.

The seasonal structure of electricity demand is remarkably strong and consistent. The daily, weekly, and yearly patterns we saw in the EDA are not subtle features buried in noise; they are the dominant signal in the data. The t-test confirmed what the plots already suggested: summer demand is about 270 MW higher than winter demand on average, and that difference is unambiguously statistically significant.

The stationarity tests were also informative. Knowing that RT_Demand does not have a long-run trend gave us confidence that a lag-based model would be stable over time without needing differencing, and the results bear that out.

The Random Forest model performed better than we initially expected. Achieving a MAPE of around 3 to 5 percent.

6.1 Ways to Improve the Model

There are several natural extensions we would pursue with more time. A day-of-week feature and a public holiday flag would help, since the model currently has no way to know that Saturday demand is structurally lower than Tuesday demand, or that a long weekend is coming.

On the modeling side, we would try XGBoost or LightGBM as alternatives because they tend to train faster and often outperform Random Forest on tabular data. For a more ambitious approach, an LSTM neural network could learn the multi-scale seasonality directly from the sequence without requiring manual feature engineering. We would also consider adding prediction intervals to the output, either through quantile regression or conformal prediction, because knowing the uncertainty around a forecast is just as important as the forecast itself for operational grid planning.

7. References

[1] ISO New England. (2025). System Market Data (SMD) Hourly. Retrieved from https://www.iso-ne.com

[2] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[3] Seabold, S., and Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. Proceedings of the 9th Python in Science Conference.

[4] Streamlit Inc. (2024). Streamlit Documentation. Retrieved from https://docs.streamlit.io

[5] Plotly Technologies Inc. (2024). Plotly Python Graphing Library. Retrieved from https://plotly.com/python/

[6] Dickey, D. A., and Fuller, W. A. (1979). Distribution of the Estimators for Autoregressive Time Series With a Unit Root. Journal of the American Statistical Association, 74(366), 427-431.

[7] Kwiatkowski, D., Phillips, P., Schmidt, P., and Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. Journal of Econometrics, 54(1-3), 159-178.

[8] Welch, B. L. (1947). The generalization of Student's problem when several different population variances are involved. Biometrika, 34(1-2), 28-35.

[9] Walker, S., Khan, W., Katic, K., Maassen, W., and Zeiler, W. (2020). Accuracy of different machine learning algorithms and added-value of predicting aggregated-level energy performance of commercial buildings. Energy and Buildings, 209, 109705.

[10] Ahmad, M. W., Mourshed, M., and Rezgui, Y. (2017). Trees vs Neurons: Comparison between random forest and ANN for high-resolution prediction of building energy consumption. Energy and Buildings, 121, 284-297.

[11] Swief, R. A., El-Amary, N. H., and Temraz, H. K. (2025). Seasonal forecasting of the hourly electricity demand applying machine and deep learning algorithms impact analysis of different factors. Scientific Reports, 15, 9252.

[12] Bergstra, J., and Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. Journal of Machine Learning Research, 13, 281-305.

[13] Osborn, D. R., Chui, A. P. L., Smith, J. P., and Birchenhall, C. R. (1988). Seasonality and the order of integration for consumption. Oxford Bulletin of Economics and Statistics, 50(4), 361-377.

[14] Cleveland, R. B., Cleveland, W. S., McRae, J. E., and Terpenning, I. (1990). STL: A Seasonal-Trend Decomposition Procedure Based on Loess. Journal of Official Statistics, 6(1), 3-73.